finally block : [100% Guarantee for execution regardless of exception]

```
try
{
    Database connection code;
    Database open;
    Connection established;
    making the transaction;
    Updating the records;
    connection close;
    database close;
}
catch(Exception e)
{
    IO.println("Handling the exception");
}
```

Exception encountered

SBI Database

We should never write any closing statement inside try block

* The main purpose of finally block to handle the resourse.

* If our java control enters in the try block then finally block will be always executed, even if have in exception in the try block and we don't have catch block.

* If we use the combination of **try and finally** then only the resourses will be handled but not the exception, on the other hand, if we use the combination of **try-catch and finally then exception and resourses both will be handled.**

****In What cases finally block will not be executed :**

Two cases :

a) If we write System.exit(0) and this statement is executed inside the try block.
b) If we write any infinite loop inside try block.

//Programs :

```
package com.rawi.finally_block;

public class FinallyBlockDemo1
{
    public static void main(String[] args)
    {
        IO.println("Main method started");
        try
        {
            IO.println("Try block");
            int x = 30/0;
            IO.println(x);
        }
        finally
        {
            IO.println("Finally block");
        }
        IO.println("Main method ended");
    }

}
```

Even, We have an exception in the try block but finally block will be executed

```
package com.rawi.finally_block;

public class FinallyBlockDemo2
{
    public static void main(String[] args)
    {
        IO.println("Main method started");
        try
        {
            int size = Integer.parseInt(IO.readln("Enter the size of the array :"));
            int []arr = new int[size];
        }
        catch(NegativeArraySizeException e)
        {
            IO.println("Array size must be a positive integer");
        }
        finally
        {
            IO.println("Resourses are handled here");
        }

        IO.println("Main method ended");
    }

}
```

try-catch & finally will handle the exception as well as resourse

```
package com.rawi.finally_block;

public class FinallyBlockDemo3
{
    public static void main(String[] args)
    {
        IO.println("Main method started");
        try
        {
            System.exit(0);
            int x = 10;
            int y = 0;
            int z = x/y;
            IO.println(z);

        }
        finally
        {
            IO.println("Finally block");
        }

    }

}
```

Here finally block will not be executed

Limitation of finally block :

The following are the limitation of finally block :

1) In order to close the resourses, user is responsible to write finally block manually.

2) Due to finally block the length of the program will be increased.

3) In order to close the resourses inside the finally block, we need to declare the resourses outside of try block.[Otherwise Block level variable]

```
package com.rawi.finally_block;

import java.util.InputMismatchException;
import java.util.Scanner;

public class LimitationOfFinally {

    public static void main(String[] args)
    {
        Scanner sc = null;
        try
        {
            sc = new Scanner(System.in);
            IO.print("Enter your Salary :");
            double salary = sc.nextDouble();
            IO.println("Your Salary is :"+salary);

        }
        catch(InputMismatchException e)
        {
            System.err.println("Input is not in a proper format");
        }
        finally
        {
            IO.println("Finally Block");
            sc.close();
        }

    }

}
```

Types of exception in java :

1) Checked Exception
2) Unchecked Exception
3) Error (Serious Problem and Developr is not responsible)

*****Difference between Checked Exception and Unchecked Exception :**

Strange behaviour of our Java compiler :

1) Foo.class is already available but still javac is saying ClassNotFoundException
2) Exception always encountered at runtime then why java compiler is saying ClassNotFoundException at compile time

3) 10/0, AE but compiler is compiling the code

Checked Exception :

A checked exception is a common exception that must be declared (throws) or handled by the application code (try catch) where it is thrown. Here compiler takes very much care and wanted the clarity regarding the exception by saying that, by using this code you may face some problem at runtime and you did not report me how would you handle this situation at runtime are called Checked exception, so provide either try-catch or declare the method as throws.

Except RuntimeException, all the checked exceptions are directly sub class of java.lang.Exception OR java.lang.Throwable (except Error).

Eg:

FileNotFoundException, IOException, InterruptedException,ClassNotFoundException, SQLException, CloneNotSupportedException, EOFEXception and so on

Unchecked Exception :-

An unchecked exception is rare and any exception that does not need to be declared or handled by the application code where it is thrown, here compiler does not take any care are called unchecked exception.

Unchecked exceptions are directly entertain by JVM because they are rarely occurred in java.[And known as RuntimeException]

Error, sub classes of Error, RuntimeException and its sub classes are Unchecked Exception.

Eg:

ArithmeticException, ArrayIndexOutOfBoundsException, NullPointerExceptiion, NumberFormatExceptiion, ClassCastExceptiion, ArrayStoreExceptiion and so on.

Some Bullet points regarding Checked and Unchecked :

Checked Exception :

1) Common Exception
2) Compiler takes care (Will not compile the code)
3) Handling is compulsory (try-catch OR throws)
4) Directly the sub class of java.lang.Exception OR java.lang.Throwable (Excluding RuntimeException and Error)

Unchecked Exception :

1) Rare Exception
2) Comiler will not take any care
3) Handling is not Compulsory
4) Error and its sub classes, RuntimeException and its sub classes